

Food Ordering for Small Shops — Phase 1 System Design

1. Problem Statement

Small food shops (tea stalls, breakfast counters, snack shops) take orders manually, causing delays, order mix-ups, and zero data on what sells. This product lets a customer scan a QR at the shop, see a live menu, and place an order without calling the shopkeeper — while the shopkeeper manages their own menu in real time from an admin login.

Why it exists: faster ordering, fewer errors, and (later) sales data the shopkeeper never had before.

Users:

- **Vendor (Shopkeeper)** — owns a shop profile, logs in via a vendor-specific QR/link, updates menu availability and prices.
- **Customer** — walk-in, no account, scans a shop QR, enters only their name, orders from the live menu.

2. Roles & Access Model (this is the core design decision)

Two separate QR codes per shop, two separate flows:

	Vendor QR	Customer QR
Who scans	Shopkeeper	Walk-in customer
Leads to	Login screen → Admin dashboard	Menu screen (no login)
Auth	Yes (phone/email + password or OTP)	None — just a name field
Can do	Add/edit/remove items, mark in-stock/out-of-stock, set price, view incoming orders	Browse menu, add to cart, place order, see order status

This means every shop has: 1 vendor account, 1 vendor QR, 1 customer-facing QR, its own menu, its own order queue.

3. Feature List

MVP (Phase 1)

- Vendor signup/login

- Vendor: add/edit/delete menu items (name, category, price, veg/non-veg tag, image optional, in-stock toggle)
- Vendor-specific QR generation (auto-created on signup)
- Customer-facing QR → menu view (no login)
- Customer enters name only → places order
- Order goes to vendor's live order queue
- Order status: Placed → Preparing → Ready
- Basic order history per shop (for later analytics)

Nice to Have (Phase 2 candidates)

- Order status visible to customer in real time (poll or push)
- Multiple categories/tabs in menu (Beverages, Breakfast, Meals, Snacks)
- Table number / token number instead of just name, if shop has seating
- Vendor dashboard: today's orders, revenue, best-seller

Future

- Business analytics dashboard (peak hours, top items, repeat visit patterns)
- Multi-shop chains / franchise support
- Payments integration
- Customer-side order history (would require some identifier — conflicts with your no-phone-number rule, needs a decision later)

4. Functional Requirements

- Vendor can sign up and log in to a shop-specific admin panel.
 - Vendor can add a menu item with name, category, price, and availability.
 - Vendor can toggle an item in-stock/out-of-stock instantly (e.g., "tea" runs out at 5pm).
 - System auto-generates one QR for vendor login and one QR for customer ordering, both tied to the shop ID.
 - Customer scans the customer QR, sees only in-stock items, enters their name, and places an order.
 - Vendor sees new orders appear in a live queue with the customer's name and items.
 - Vendor can mark an order as Preparing / Ready / Completed.
 - No customer phone number, email, or persistent account is collected or stored.
-

5. Non-Functional Requirements

- **Privacy-by-design:** no PII beyond a first name, which is not stored long-term as

identity — just attached to that single order.

- **Performance:** menu load < 1s on average mobile data (customers are scanning on shop wifi/4G, often mid-crowd).
 - **Availability:** shop-hours uptime matters more than 24/7 (99% is fine for phase 1, not 99.99%).
 - **Simplicity:** customer flow must work with zero app install — mobile web only (QR → browser).
 - **Multi-tenancy:** every query is scoped by shop_id; one shop must never see another shop's menu/orders.
-

6. User Flows

Vendor Flow

```
Scan Vendor QR / open link
    ↓
Login (phone/email + password)
    ↓
Admin Dashboard
    ↓
Add/Edit Menu Items ↔ Toggle Availability
    ↓
View Incoming Orders → Update Status
```

Customer Flow

```
Scan Customer QR at shop table/counter
    ↓
Menu Page loads (shop's live, in-stock items only)
    ↓
Enter Name (no phone number)
    ↓
Add items to cart
    ↓
Place Order
    ↓
See simple confirmation ("Order placed, look for [Name]")
```

7. Screens (Wireframe List)

Vendor side: Login → Dashboard (order queue) → Menu Manager (add/edit item) → QR

page (view/download both QRs) **Customer side:** Menu (categorized: Beverages / Breakfast / Meals / Snacks) → Cart → Name entry → Order confirmation

8. Database Design

Shop

- shop_id (PK), name, owner_vendor_id, customer_qr_token, vendor_qr_token, created_at

Vendor

- vendor_id (PK), shop_id (FK), phone/email, password_hash

MenuItem

- item_id (PK), shop_id (FK), name, category, price, veg_flag, in_stock (bool), image_url (optional)

Order

- order_id (PK), shop_id (FK), customer_name, status (placed/preparing/ready/completed), created_at

OrderItem

- order_item_id (PK), order_id (FK), item_id (FK), quantity, price_at_order_time

Note: no Customer table with persistent identity — `customer_name` lives only on the Order row. This is intentional given your privacy requirement, but flag it now: it means no way to recognize a *repeat* customer in Phase 1. That's fine for MVP, but will need a rethink (e.g. anonymous device token) if Phase 2 analytics wants "repeat visit rate."

9. API Design (representative, not exhaustive)

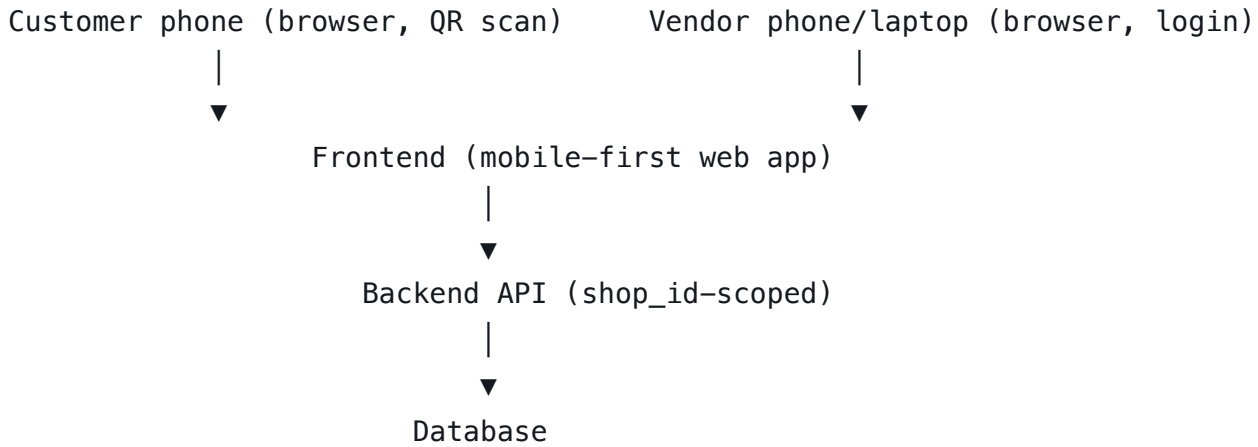
```
POST /vendor/signup
POST /vendor/login
GET  /vendor/{shop_id}/menu
POST /vendor/{shop_id}/menu
PUT  /vendor/{shop_id}/menu/{item_id}    # edit / toggle stock
GET  /vendor/{shop_id}/orders            # live queue
PUT  /vendor/{shop_id}/orders/{order_id} # update status

GET  /shop/{customer_qr_token}/menu      # public, no auth
POST /shop/{customer_qr_token}/order     # {customer_name, items[]}
```

GET /order/{order_id}/status
screen

public, for confirmation

10. Basic Architecture



Optional additions once volume grows: Redis for live order queue push, image storage (S3-like) for menu photos.

11. Capacity Estimation (rough, phase 1)

- Target: single-shop or handful-of-shops pilot, not city-scale.
- Concurrent customers per shop: 5-20 at a time (small shop foot traffic).
- Orders/day/shop: 50-300.
- DB size: trivial at this stage (a few thousand rows/month/shop) — no special scaling needed yet.

12. Security & Privacy

- Vendor auth: password hash + session/JWT, scoped to their shop_id only.
- Customer side: no auth, but the `customer_qr_token` should be shop-specific and non-guessable, so no one can hit another shop's menu/order endpoint by editing a URL.
- No phone numbers, no persistent customer accounts — document this explicitly as a data-minimization decision (matters for analytics scope later too).
- HTTPS everywhere, input validation on order submission (rate-limit order creation from the public endpoint to prevent spam orders).

13. Deployment

- Frontend: static/mobile-web hosting (Vercel/Netlify-style)
 - Backend: single API service (can start on Render/Railway/similar)
 - Database: managed Postgres/MySQL
 - QR codes: generated server-side per shop, downloadable as PNG for the shopkeeper to print
 - SSL: automatic via hosting provider
-

14. Deferred to Later Phases (flagged, not designed yet)

- Business analytics (top items, peak hours, revenue trends) — needs Order/OrderItem data model above, which is already analytics-ready even though the dashboard isn't built yet.
 - Payments
 - Push/real-time order status for customers
 - Any form of customer recognition — blocked by your no-phone-number rule until you decide on an alternative (e.g. anonymous local storage token).
-

One-Page Flow

```
Problem Statement → Roles & Access Model → Feature List (MVP split)
    ↓
Functional Reqs → Non-Functional Reqs
    ↓
Vendor Flow + Customer Flow (two QR paths)
    ↓
Screens → Database Design → API Design
    ↓
Architecture → Capacity → Security & Privacy
    ↓
Deployment → Start Development
```